

0x04 Что такое модуль ядра. Как его написать, собрать, запустить

курс «Разработка ядра Linux»



идет ли запись ?

Меня хорошо слышно?



Маршрут вебинара



Правила вебинара



Активно
участвуем



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



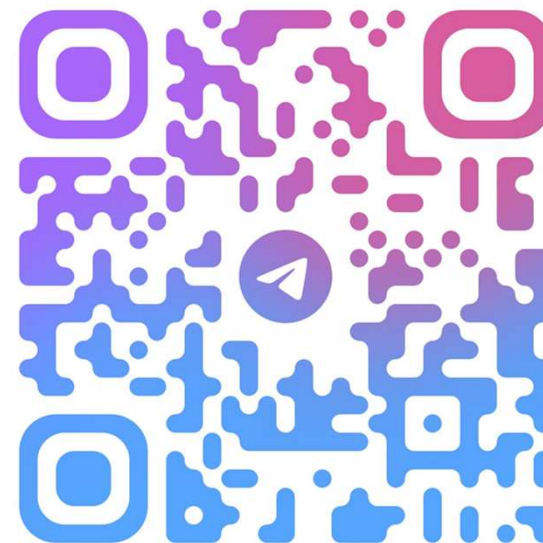
Правила вебинара



Off-topic обсуждаем
в закрытом чате в
Telegram !

***(также обсудить Off-topic будет**
возможность в конце занятия после
отключения записи)

Кто еще не присоединился,
сканируйте QR код :)



Отзывы

Вебинары во многом адаптивны.

Мы наблюдаем за вашим обсуждением в Telegram, берем в работу идеи из ваших отзывов на занятия и домашние задания и становимся лучше!



Пожалуйста, заполняйте опросы после занятий и ДЗ, мы читаем все ваши сообщения :)



Цели вебинара

К концу занятия вы ...

1. Научитесь писать код к простым модулям ядра
2. Узнаете основные функции и утилиты для разработки в "kernel space"
3. Поймете, как модули ядра загружаются и исполняются
4. Узнаете как посмотреть что модуль работает и что такое буфер сообщений ядра



Ключевые моменты

Что такое модули ядра?

- Это куски кода, которые можно динамически подгружать в ядро, чтобы расширять его функциональность без перезагрузки
- *Например, драйверы устройств*

Без них пришлось бы собирать монолитные ядра размером с роман «Война и мир» — и перезагружаться каждый раз, когда нужно добавить новую фичу



Hello, Kernel World!

Традиция жива: первый модуль выводит "Hello world".

На следующем занятии будет приведен пример кода, который загружается в ядро и скромно здоровается.

Предупреждение: если после этого ваш компьютер превратится в кирпич — вы что-то сделали не так (но зато научились перезагружаться!).



Особенности работы с модулями

- **Версионирование:** Модуль, скомпилированный для одного ядра, может не заработать на другом
 - *Решение:* отключить CONFIG_MODVERSIONS или молиться богам совместимости
- **Инструменты:**
 - lsmod (показать загруженные модули)
 - insmod (загрузить модуль)
 - modprobe (загрузить с зависимостями)
 - rmmod (выгрузить модуль)
 - dmesg (читать сообщения ядра)



Полезные утилиты

dmesg и логирование

Основные флаги для **dmesg**

Флаг	Описание
-H	Показывает человекочитаемый формат вывода (полное форматирование с цветовой палитрой).
-T	Показывает человекочитаемые временные метки (только дата и время).
-w	Режим реального времени (аналог tail -f).
-l	Фильтр по уровню логирования (например, -l err).
-k	Показывает только сообщения ядра.



Уровни логирования в ядре

```
pr_info("Сообщение уровня INFO"); //printk(KERN_INFO "Сообщение уровня  
INFO");  
pr_err("Сообщение уровня ERROR"); //printk(KERN_ERR "Сообщение уровня  
ERROR");
```

Уровни приоритета:

EMERG, ALERT, CRIT, ERR, WARNING, NOTICE, INFO, DEBUG.



Также в ядре Linux существует семейство подобных макросов для разных уровней логирования:

- `pr_emerg()` — для сообщений уровня KERN_EMERG (самый высокий приоритет)
- `pr_alert()` — KERN_ALERT
- `pr_crit()` — KERN_CRIT
- `pr_err()` — KERN_ERR
- `pr_warn()` — KERN_WARNING
- `pr_notice()` — KERN_NOTICE
- `pr_debug()` — KERN_DEBUG (обычно используется только при включённой отладке)



printk_ratelimit – это

функция в ядре Linux, которая ограничивает частоту вывода сообщений с помощью `printk`, чтобы предотвратить избыточное логирование и возможное засорение системного журнала или консоли



Основные особенности

printk_ratelimit

- Возвращает 1 (true), если текущий вывод сообщений не превышает заданный лимит, и 0 (false), если лимит превышен
- Лимит по умолчанию — не более 10 сообщений каждые 5 секунд
- Лимит задаётся через параметры:
 - `/proc/sys/kernel/printk_ratelimit_burst` — максимальное количество сообщений за период
 - `/proc/sys/kernel/printk_ratelimit` — минимальный интервал между блокировками вывода (обычно 5 секунд)
- Если лимит превышен, дальнейшие сообщения временно подавляются, чтобы избежать перегрузки логов и консоли



**modinfo, lsmod &
insmod/rmmod/modprobe**

modinfo:

Информация, выводимая modinfo, включает:

1. **filename:** Путь к файлу модуля.
2. **license:** Лицензия модуля.
3. **description:** Краткое описание модуля.
4. **author:** Автор модуля.
5. **depends:** Список зависимостей модуля.
6. **vermagic:** Версия ядра, для которой предназначен модуль.
7. **parameters:** Список параметров модуля.



Эта информация полезна для разработчиков и администраторов при работе с модулями ядра, позволяя им понимать функциональность и требования модулей



Ismod:

- Ismod: Список загруженных модулей

Эта команда отображает таблицу с информацией о модулях, где каждый модуль представлен строкой с тремя столбцами: имя модуля, размер и список модулей, которые используют его.



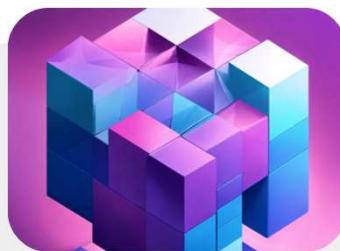
insmod/rmmod/modprobe:



insmod

Загрузка модулей по пути к файлу с расширением ".ko".

Эта команда загружает модуль по указанному пути к файлу модуля. Если модуль зависит от других модулей, которые еще не загружены, insmod выдаст ошибку и не загрузит модуль.



rmmod

Выгрузка конкретного модуля по его имени.

Эта команда выгружает модуль по его имени. Если модуль используется другими модулями, выгрузка не произойдет.



modprobe

Загрузка и выгрузка модулей с их зависимостями.

Эта команда загружает и выгружает (с флагом -r) модуль по его имени и автоматически подгружает и выгружает все необходимые зависимости.

Основные отличия

Параметр	insmod/rmmod	modprobe
Зависимости	Не обрабатывает зависимости. Требуется ручной загрузки всех зависимостей.	Автоматически разрешает и загружает зависимости модуля.
Путь к модулю	Требуется полный путь к файлу <code>.ko</code> (например, <code>/path/to/module/module.ko</code>).	Работает с именами модулей (Но только в каталоге <code>/lib/modules/\$(uname -r)/kernel/</code>).
Конфигурация	Не использует конфигурационные файлы.	Читает настройки из <code>/etc/modprobe.d/</code> (но требует обновления зависимостей).
Выгрузка	<code>rmmod</code> выгружает только указанный модуль (если нет зависимостей).	<code>modprobe -r</code> выгружает модуль и его неиспользуемые зависимости.
Безопасность	Может вызвать ошибки при отсутствии зависимостей или конфликтах.	Более устойчив к ошибкам благодаря обработке зависимостей.



Проблемы при загрузке и выгрузке модулей

- **Зависимости**

- Если модуль зависит от других модулей, которые еще не загружены, загрузка не произойдет
- Используйте `modprobe` для автоматического решения зависимостей

- **Использование модуля**

- Если модуль используется другими модулями или системой, его выгрузка невозможна до тех пор, пока все зависимости не будут выгружены



Написание и сборка модулей ядра

Написание и сборка модулей ядра

Для написания модуля ядра необходимо создать файл с исходным кодом на языке C, который будет компилироваться в объектный файл с расширением **.ko**.

Пример кода модуля (hello_world.c)

Пример кода модуля (hello_param.c)

Пример кода модуля (hello_param_cb.c)



Немного про module_param

S_IRWXU — права rwx для владельца (User)=0700

S_IRWXG — права rwx для группы (Group)=0070

S_IRWXO — права rwx для остальных (Others)=0007

S_IRUSR — права r.. для владельца (User)=0400

S_IRGRP — права r.. для группы (Group)=0040

S_IROTH — права r.. для остальных (Others)=0004

и т.д.



Сборка модулей (Makefile & Kbuild)

Пример Makefile

```
PWD := $(shell pwd)
KERNEL_DIR ?= /lib/modules/$(shell uname -r)/build

SRC_DIR := $(PWD)/src
BUILD_DIR := $(PWD)/build
BUILD_FILES += $(SRC_DIR)/*.ko

CLANG_FORMAT_VERS ?= 19
CLANG_FORMAT := clang-format-$(CLANG_FORMAT_VERS)
CLANG_FORMAT_FLAGS += -i
FORMAT_FILES := $(SRC_DIR)/*.c

$(shell mkdir -p $(BUILD_DIR))

kbuild:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules
    mv $(BUILD_FILES) $(BUILD_DIR)/.

format:
    $(CLANG_FORMAT) $(CLANG_FORMAT_FLAGS) $(FORMAT_FILES)

clean:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean

.PHONY: kbuild clean format
```



Пример Kbuild:

```
obj-m += src/hello_world.o  
obj-m += src/hello_param.o  
obj-m += src/hello_param_cb.o
```



Разница между obj-m и obj-y

obj-m (Module):

Указывает, что объект должен быть собран как **загружаемый модуль** (.ko).

Пример:

```
obj-m += my_module.o  # →  
my_module.ko
```

obj-y (Yes):

Включает объект в **состав самого ядра** (статическая линковка).

Пример (в ядре Linux):

```
obj-y += core/  # Собирает код из  
папки core и включает его в ядро
```

Итог:

- obj-m — для модулей
- obj-y — для кода, встраиваемого в ядро.



Best Practices:

- **Проверка ошибок:** Всегда проверяйте возвращаемые значения.
- **Документация:** Используйте `MODULE_DESCRIPTION()`, `MODULE_VERSION()`.
- **Безопасность:** Избегайте утечек памяти.
- **Совместимость:** Учитывайте версию ядра.



ИТОГ...

Помните:



«С модулями ядра можно сделать всё, но не стоит этого делать» (с)
Народная мудрость.

Удачи в экспериментах!  

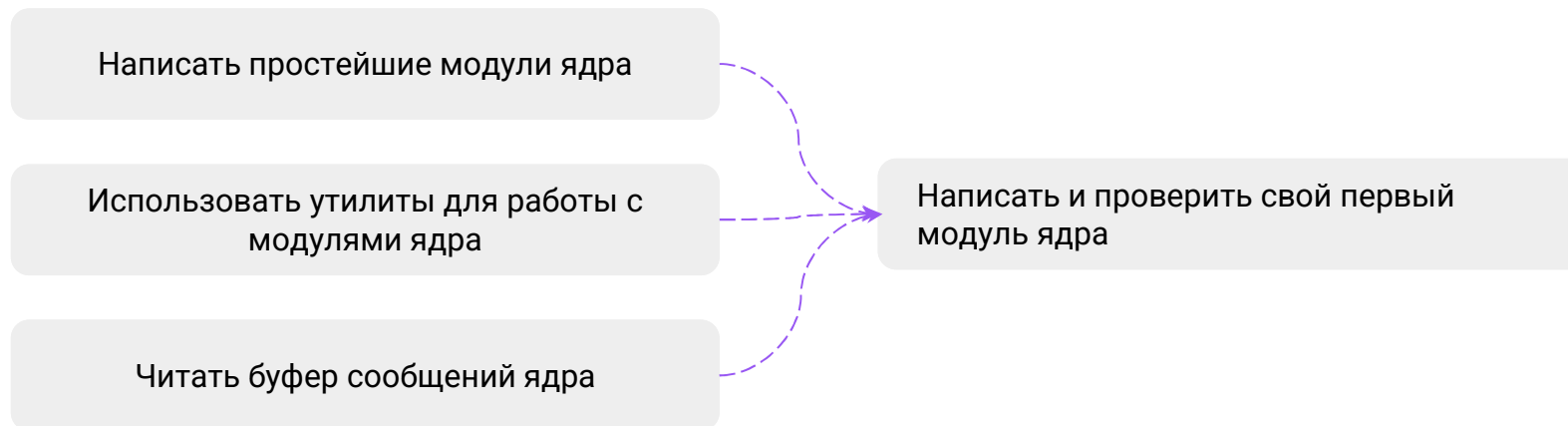


«No rain - no gain» (с) английская пословица.



Теперь вы сможете

Проверим достижение целей



Вопросы (по лекции)



если есть вопросы



если вопросов нет

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️ ❤️

Приглашаем на следующий вебинар: “Связанные списки в ядре Linux. Реальные примеры из кода ядра Linux”



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой

Что необходимо повторить ?

- 1) Связные списки
- 2) Makefile, Kbuild
- 3) Указатели в языке Си.



обсуждаем Off-topic

***Время отключить запись**

otus

